

Fine-tuning vs. retrieval: a Yu-Gi-Oh small language model

Harman Singh Sandhu - Build an Enterprise SLM from Scratch - August 2026

What I built

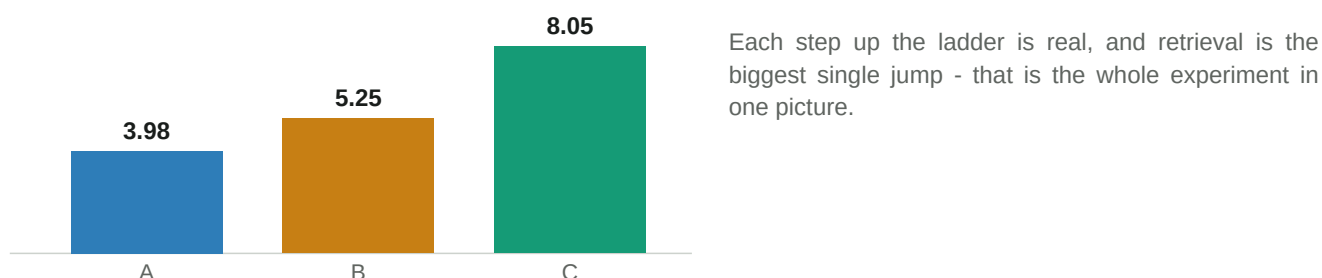
I built three versions of a Yu-Gi-Oh question-answering system on top of Gemma 2 2B and compared them on the same 60 held-out questions. System A is the untouched base model, closed-book; System B is my QLoRA fine-tune of the same model, also closed-book; and System C is that fine-tune with a retriever feeding it passages from my own corpus. I picked Yu-Gi-Oh rulings and card facts on purpose: the base model clearly does not know them - it scored about 1.5 out of 12 on a quick probe and confidently made up card text - so the three systems can actually separate. Every answer is graded by a reference-grounded AI judge that compares against a written gold answer and its exact evidence, so the scores are reproducible by someone who does not play the game.

Headline result (mean score / 10, n = 60)

base, closed-book	3.98 +/- 0.39	-
fine-tune, closed-book	5.25 +/- 0.54	+1.27 over A, p = 0.007 (significant)
fine-tune + retrieval	8.05 +/- 0.42	+2.80 over B, p < 0.001 (significant)

Paired bootstrap 95% CI + t-test + Wilcoxon, on the same items.

The one plot I would show



What I expected, and where it disagreed

Before I ran it I expected that fine-tuning closed-book would not really beat the base model, because teaching a model question-answer pairs teaches it the shape of an answer, not the facts. Retrieval matched that expectation - it was the biggest win by far. But one result surprised me: my fine-tune did significantly beat the base model closed-book (+1.27). When I broke the score down, the reason was groundedness, which went from 0.18 up to 0.87. The base model gives long, confident, wrong answers; my fine-tune learned to give tight, careful ones that do not invent card details, and the judge rewards that. The facts still only come from retrieval - correctness only jumps from 2.35 to 3.85 when I turn retrieval on - so the fine-tune learned a better answer shape, not new facts. The thesis held; the fine-tune just helped a little more than I expected.

What cost me the most time, and what I would change

The corpus cost me the most time. My first instinct was to grab card text, but a lot of that is short, repeated effect text, and my cleaning kept either throwing away good rulings or keeping junk. I lost the most hours tailoring the cleaning to what my corpus actually held - for example I found a rare set of struck-through 'previously official' rulings that my markup stripper was silently keeping as if they were still current. Next time I would read a few hundred real pages before writing a single cleaning rule, and I would separate 'commentary about cards' from 'the cards themselves' from day one, instead of discovering halfway through that my retriever could not answer 'what does this card do' because I had never put the card facts in.

What I learnt

Three things stuck with me. First, **RAG's ceiling is the reader, not the retriever**. When I tested 'what is the effect of Blackwing Full Armor Master?' live, the retriever handed my model the exact card text and it still scored only 6 out of 10 - it named the card's type and material, but dropped the actual effect. Recall is already 0.93, so the limit is my 2.6-billion model finishing what it is handed, which is why System C stops near 8, not 10. Second, **a reference-grounded judge can grade a domain it does not know**. On that same question it scored the two made-up answers 1 out of 10 with groundedness zero, purely by checking them against the evidence I hand it - never needing to know the card itself. Third, **match the retriever to the domain's shape**. Yu-Gi-Oh questions turn on exact card names that dense embeddings blur, so pairing them with plain keyword search lifted recall@5 to 0.93 - and that hybrid is what feeds System C.